

Non-Functional Test Cases: Maintainability

Library Management System

Shared and Balanced (Coders and Non-Coders/Specification Experts)

June 5, 2025

Note: These test cases are designed to assess the maintainability of the software and its associated documentation. They are primarily based on static review and analysis techniques. Refer to the SRS and lecture materials for complete requirements and context.

IV.D Non-Functional Test Cases: Maintainability

Table 1: Maintainability Test Cases

Test Case ID	Test Title	Test Steps (Review Actions)	Expected Results (Maintainability Attributes)
Sub-Section: Code-Level Maintainability (Static Analysis)			
TC_MAINTAIN.CODE_001	Verify Code Modularity and Structure	- Review the overall project structure, including the separation of client and server code. - Inspect the source code to assess if functionalities are encapsulated in distinct, well-defined modules or components.	- The codebase is well-structured and modular. - Code exhibits high cohesion within modules and low coupling between them.
TC_MAINTAIN.CODE_002	Verify Code Readability and Coding Standards	(Coders) Lead a technical review of the code for adherence to defined coding standards. (Non-Coders) Participate in the review, focusing on overall readability and understandability. - Check for consistent naming conventions, formatting, and indentation.	- The code is readable and easy to understand. - Agreed-upon coding standards are followed consistently throughout the project. - No variables with confusingly similar names are present.
TC_MAINTAIN.CODE_003	Verify Adequacy and Clarity of Code Comments	- Identify complex logic, algorithms, or non-obvious code sections. - Review the comments associated with these sections.	- Code is commented where necessary to explain complex logic. - Comments are clear, concise, and accurately describe the code's intent and functionality.
TC_MAINTAIN.CODE_004	Verify Control Flow and Logic Integrity	- Inspect conditional statements (if-else, switch) to ensure all cases are handled. - Review all loops to verify they have clear termination conditions. - Check for off-by-one errors in loops or array indexing.	- Multiway branches cover all cases or have a proper default case. - All loops will terminate under expected operating conditions. - No off-by-one errors are present in iterations or indexing.
Continued on next page			

Table 1 – continued from previous page

Test Case ID	Test Title	Test Steps (Review Actions)	Expected Results (Maintainability Attributes)
TC_MAINTAIN.CODE.005	Verify Inter-Module Interface Correctness	• Review function/API calls between different modules (e.g., client to server, server to database). • Check the number, data types, and order of parameters/arguments passed.	• The number of input parameters equals the number of arguments passed. • The attributes (type, order) of parameters and arguments match. • Global variable definitions are consistent across modules that use them.
TC_MAINTAIN.CODE.006	Verify Ease of Deployment and Environment Management	• (Coders) Review the Docker configuration files and any deployment scripts. • Assess the process required to set up a new development or testing environment.	• The use of Docker facilitates easier deployment and environment management. • Configuration of environment variables (e.g., database URI, session secret) is straightforward and well-documented.
Sub-Section: Documentation-Level Maintainability			
TC_MAINTAIN.DOC.001	Verify Quality of Requirements and Design Documentation	• (Non-Coders) Lead a review of the SRS document. • Check for clarity, consistency, completeness, and lack of ambiguity in the requirements.	• Project documentation is clear, consistent, complete, and accurate. • Requirements are testable and clearly defined.
TC_MAINTAIN.DOC.002	Verify Quality of Test Documentation	• Review the master Test Plan and all associated test case documents (functional, non-functional, etc.). • Verify alignment between test cases and the SRS.	• The Test Plan is comprehensive and outlines a clear strategy. • Each test case includes a clear definition of the expected output or result. • Test cases are traceable back to specific requirements in the SRS.
TC_MAINTAIN.DOC.003	Verify Technical Accuracy of All Documentation	• (Coders) Review all project documents (SRS, Test Plan, Final Report) for technical accuracy. • Pay special attention to descriptions of API endpoints, data models, and system architecture.	• All documentation is technically accurate. • Sections detailing technical testing efforts and system implementation are correct.